SymQ: Exact Equivalence Checking of Reversible Boolean Circuits with a Native Monomial-ZDD Engine — an Evaluation with Matched Drivers

[TODO: author names, affiliations and contact
to be supplied and approved by the authors;
no author is listed in this draft]

Draft of 27 October 2026 — not submitted

Abstract

We study exact functional equivalence checking of classical reversible circuits built from X, CNOT, Toffoli and multi-controlled X gates, by symbolic simulation in the Boolean ring $\mathrm{GF}(2)[x_1, \ldots, x_n]/(x_i^2 - x_i)$: every output wire is kept in algebraic normal form (ANF), two circuits are equivalent iff their output polynomials coincide, and a minimum-degree monomial of a differing output is a counterexample that we re-check by concrete execution. The representation, zero-suppressed decision diagrams (ZDDs) over monomial families, is not new: PolyBoRi uses it for Boolean Gröbner bases. Our contribution is engineering and measurement. We describe symq, an open-source checker with a small native C engine (hash-consed monomial ZDD, three-product multiplication, lossless operation cache, monomial counts maintained at node creation, explicit time/node/term budgets), and an evaluation protocol that separates the engine from the driver that feeds it gates and controls the state of the caches. We find that this separation matters: earlier ratios from our own harness mixed a fused C gate loop with per-gate Python calls and measured PolyBoRi with a warm operation cache. Under identical Python drivers and cold caches, on the 19 supported reversible circuits of the MQT QCEC test suite, the native engine computes the symbolic outputs 4.16× faster than PolyBoRi (geometric mean, 95% CI [3.68, 4.67], 19/19 circuits) and 3.86× faster than CUDD BDDs; with warm caches the gap to PolyBoRi is 2.11×. Two of our own optimisations do not pay: sorting the factors of each product by monomial count *slows* the engine down (0.85×), and the gate loop fused in C, which also sorts, is no faster than a plain Python loop over the same C operations (0.97×, 95% CI [0.89, 1.06]). Output polynomials are identical across all modes and across seven variable orders per circuit. [PENDING, not a result: blind-corpus sentence, written after sealing.] Limits: at most 64 variables; ANF blows up on multipliers under every variable order we tried, and on larger adders under the default order; Python drivers for every tool; one machine; mostly sub-millisecond medians; MQT QCEC decides a different (unitary) question and is reported for context only. We claim no new algorithm and no general superiority.

1 Introduction

Reversible circuits over X, CNOT (CX), Toffoli (CCX) and multi-controlled X (MCX) gates are the classical core of many quantum algorithms (arithmetic, oracles, uncomputation) [31, 11]. Their equivalence is a purely Boolean question: two circuits on n wires are equivalent iff they compute the same permutation of $\{0,1\}^n$. Tools from quantum design automation decide it among many other questions [6], and classical techniques (BDDs [5], SAT, algebraic methods) apply directly [30].

A particularly simple exact method is symbolic simulation in the Boolean ring: each wire carries a polynomial over GF(2) in algebraic normal form (ANF), a gate with controls C and target t updates $f_t \leftarrow f_t + \prod_{c \in C} f_c$, and canonicity of the ANF turns equivalence into equality of polynomials (Section 3). The cost is the size of the intermediate polynomials. Storing them as zero-suppressed decision diagrams [20] of their monomial families shares common sub-families, and is exactly what PolyBoRi does for Boolean Gröbner-basis computations [3].

This paper does *not* propose a new representation or a new decision procedure. It asks an engineering question and tries to answer it without fooling ourselves: *how much does a small, specialised native engine gain over mature native libraries on this task, once the driver and the caches are controlled?* Our contributions are:

1. **An exact, budgeted native engine** (Section 4) for monomial ZDDs specialised to reversible-circuit

simulation, with concretely re-validated counterexamples, and its integration in an open-source checker with interchangeable exact backends.

2. **A matched-driver, cache-controlled protocol** (Section 5) that runs every engine under the same minimal Python driver and separates setup, symbolic computation, comparison, witness extraction and explicit-monomial extraction, in cold and warm cache regimes, with paired statistics and a pre-registered blind corpus.

3. **Measurements, including adverse ones** (Section 6): the gains that survive matching, the ones that do not (fused loop, factor sorting), and the families where the monomial representation fails.

The phase-sensitive extension of `symq` to measurement-based uncomputation is a separate, unfinished line of work and is not evaluated here (Section 7).

2 Background and related work

Boolean polynomials and ZDDs. ZDDs [20] are reduced, ordered decision diagrams whose nodes with a 0 high child are suppressed; they represent families of sparse sets compactly and canonically [22]. Minato already used them to manipulate polynomials implicitly [21]. Representing a Boolean polynomial by the ZDD of its monomial family, with ring operations implemented as recursive ZDD operations, is the core of PolyBoRi [3], maintained today as BRiAl [26] and reachable from Python through a modular SageMath distribution [27]; its authors applied it to formal verification of circuits [4]. Our engine uses the same representation and the same kind of recursion; it differs in scope (at most 64 variables, one operation pattern), in engineering choices (Section 4), and in being budgeted. BDDs over Shannon cofactors [5], as in CUDD [25] (used here through `dd` [9]), represent the same Boolean functions without exposing their ANF; CUDD's ZDD products operate on covers, not on GF(2) sums, so CUDD offers no ANF ring.

Algebraic verification of arithmetic circuits. Computer-algebra verification of gate-level multipliers fights the monomial blow-up that we also observe, with backward rewriting and vanishing-monomial removal [19], combinations with SAT [15], or linear relations extracted from Gröbner bases [14]. Our forward simulation uses none of these techniques and is not competitive on multipliers; we do not compare with these tools, which target And-Inverter Graphs rather than reversible circuits.

Equivalence checking of reversible and quantum circuits. Equivalence of reversible circuits has been decided with decision diagrams and SAT [30]; RevLib [31] provides benchmarks. For quantum circuits, decision-diagram checkers go back to QuIDD [28]; MQT QCEC [6, 7, 23] combines decision-diagram, simulation and ZX checkers. On X/CX/MCX circuits, unitary equality coincides with functional equality, so QCEC is a natural point of comparison even though it answers a more general question and builds no polynomial. Path sums [1] carry, for reversible gates, exactly our ANF outputs together with a phase polynomial; phase-based methods optimise and check larger gate sets [2]. QuPRS combines path-sum reduction with weighted model counting [13], and FeynmanDD reduces path-integral questions to counting with classical decision diagrams [29]. Closest to our representation, Irene [16] checks hybrid quantum programs, represents Boolean expressions as XOR–AND graphs rather than ANF, and reports, counting ANF monomials with ZDDs, that explicit ANF would need more than 6 GiB on 30 of its program pairs: independent evidence that ANF is the wrong representation for some workloads. Measurement and feed-forward are handled by dedicated checkers [12, 8, 24] and by outcome-complete stabilizer simulation [17, 18]; they are outside our scope.

What is new here. `paper/CLAIMS.md` lists each claim with its prior art and the experiment that could refute it. In short: the representation and the product recursion are known; a specialised, budgeted native engine and, above all, an evaluation that controls driver and cache effects are what we contribute. We found no prior evaluation of a monomial-ZDD engine for reversible-circuit equivalence against PolyBoRi and CUDD under matched drivers, but our search (arXiv full-text queries and the references above) is not exhaustive.

3 Problem and semantics

Definition 1 (Circuits)**.** A circuit of width n is a list of gates (C, t) with $C \subseteq \{0, \ldots, n-1\}$ and $t \notin C$. On a state $s \in \{0,1\}^n$ the gate sets $s_t \leftarrow s_t \oplus \prod_{c \in C} s_c$ (X: $C = \emptyset$; CX: $|C| = 1$; CCX: $|C| = 2$; MCX otherwise).

Let $B_n = \mathrm{GF}(2)[x_0, \ldots, x_{n-1}]/(x_i^2 - x_i)$. Each element has a unique ANF, a sum of distinct square-free monomials, identified with a family of subsets of $\{0, \ldots, n-1\}$; addition is symmetric difference of families.

Fact 1 (canonicity)**.** *B_n is isomorphic to the ring of Boolean functions $\{0,1\}^n \to \{0,1\}$; two polynomials denote the same function iff their monomial families are equal.*

Symbolic simulation starts from $f_i = x_i$ and applies $f_t \leftarrow f_t + \prod_{c \in C} f_c$ for each gate.

Proposition 1 (soundness and completeness)**.** *For every input a, $(f_0(a), \ldots, f_{n-1}(a))$ is the circuit's output on a. Hence two circuits are equivalent iff their output families are equal wire by wire.*

Proof. Induction on gates: evaluation at a is a ring homomorphism $B_n \to \mathrm{GF}(2)$, so $(f_t + \prod f_c)(a) = s_t \oplus \prod s_c$; combine with the fact. □

Proposition 2 (minimum-degree witness)**.** *If $p \neq 0$ and M is a monomial of p of minimum degree, then $p(\chi_M) = 1$, where χ_M sets exactly the variables of M to 1.*

Proof. $p(\chi_M)$ is the parity of the number of monomials of p contained in M. A monomial strictly contained in M would have smaller degree, so only M itself counts. □

Applied to $f_i^L + f_i^R$ for the first differing output i, Proposition 2 yields an input on which the circuits differ. `symq` re-executes both circuits on it and raises an internal-inconsistency error if they agree, so a faulty backend can never return an unchecked counterexample. Equivalence verdicts have no such independent check; they rest on the propositions above and on differential testing (Section 5). Full statements, including budgets, are in `paper/FORMAL_MODEL.md`; none of them is machine-checked.

4 The native engine

Representation. Nodes are triples (ℓ, lo, hi) in a single array, hash-consed through an open-addressing unique table; node 0 is the empty family (0), node 1 is $\{\emptyset\}$ (1). A node at level ℓ with variable $v = x_{\pi(\ell)}$ denotes $lo + v \cdot hi$; zero suppression forbids $hi = 0$; both children lie at strictly larger levels (or are terminals), which is what makes the representation canonical. Children are always created before their parent, so they have smaller identifiers, and every bottom-up quantity (monomial count, minimum-degree monomial) is computed by a single pass in identifier order, without recursion.

Operations. Addition is the symmetric-difference recursion. Multiplication splits both operands on the top variable v, with $P = P_0 + vP_1$ and $Q = Q_0 + vQ_1$:

$$PQ = P_0Q_0 + v\,(P_0Q_1 + P_1Q_0 + P_1Q_1), \tag{1}$$

which follows from $v^2 = v$. Terminal cases include $P \cdot P = P$ (every element of B_n is idempotent) and $P + P = 0$. An algebraically equal two-product form, $v((P_0 + P_1)(Q_0 + Q_1) + P_0Q_0)$, built up to two orders of magnitude more nodes on one of our circuits under a reversed variable order, so we use (1); we make no asymptotic claim about either form.

Engineering choices. (i) A lossless operation cache: entries are never evicted and the table grows, unlike the lossy computed tables of general-purpose packages; managers are short-lived (one per verification), so no garbage collection is performed. (ii) Monomial counts are stored per node at creation as signed 64-bit integers. A 64-variable family can have up to 2^{64} monomials, and in the measured version such a count can overflow, which is undefined behaviour in C (with the compiler used, in practice a wrap-around that no check detects); we found this defect while preparing this paper. Its fix (saturating counts, and an error when a saturated polynomial is materialised) is under review and is *not* part of the measured code. Counts drive budgets, factor ordering, extraction and the choice of representation in `symq`'s adaptive backend, never a verdict (equality is node identity), and the largest output we measured has 8 520 monomials. (iii) Monomials are 64-bit masks, which bounds the width to 64. (iv) A `simulate` entry point takes the whole gate list (control indices, offsets, targets) and runs the loop in C, multiplying the controls of each gate in increasing monomial count. (v) Budgets on time (checked every 1 024 recursive steps and every 64 gates), nodes (at creation) and monomials (when a polynomial is materialised) raise an exception; no partial result is ever returned. The engine is about 1 240 lines of C against the CPython API and is optional: without a compiler, the pure-Python backends remain.

5 Evaluation methodology

Questions. (Q1) How fast is the native engine relative to PolyBoRi and CUDD when only the engine *and its Python binding* differ (the driver being identical)? (Q2) Which parts of the engine and driver matter (ablation)? (Q3) Does the comparison survive circuits that influenced no design decision? (Q4) Are all results exact?

Drivers. Every engine is run under the same drivers: (a) *minimal loop*: the same Python loop for all tools, which multiplies the controls of each gate in gate order and adds the product to the target, with no guard and no counter; only the two ring operations differ (for PolyBoRi, Python's `*` and `+` on `BooleanPolynomial`; for CUDD through `dd` [9], conjunction and `xor`); (b) *minimal loop, sorted*: the same with factors sorted in Python by monomial count (`symq`: its stored count; PolyBoRi: `len`); (c) *`symq`'s `Algebra` driver*, which decodes each gate and applies its guards and counters inside the timed phase; (d) for `symq` only, the fused C loop that it ships, fed control, offset and target arrays built before the timer (as the gate list of the loops is). Equality is node identity for `symq` and CUDD and `(a+b).is_zero()` for PolyBoRi (Sage's `==` compares term by term and is an order of magnitude slower on large outputs). The loop calls the engines through their Python bindings (a C extension for `symq`, Cython for PolyBoRi and `dd`), whose per-call costs differ; we measure them separately (Section 6) rather than attribute the whole loop time to the engine.

Cache regimes. SageMath caches `BooleanPolynomialRing` objects by their arguments: two rings of the same width are the same object, with the same operation cache. In our earlier harness every PolyBoRi repetition after the first therefore reused one ring and its caches (an inference from Sage's caching behaviour, which we did not measure in those runs), while every other engine started from a fresh manager. We now measure two regimes for every engine: *cold* (a fresh manager per run; for PolyBoRi, a ring with fresh variable names) and *warm* (one manager reused after a filling run, i.e. re-verification). Setup (manager and variables) is timed separately from the symbolic computation.

Corpora. (1) The 19 supported circuits of MQT QCEC's `test/circuits/original` (16–35 wires; one circuit with controlled-V gates is rejected), imported byte for byte with hashes and license [23]. They have influenced our design, so they are not a holdout. (2) A *blind* corpus of 30 circuits generated by a seeded script and frozen by a hash manifest before any symbolic measurement: Cuccaro adders [10] and controlled adders up to 31 bits, shift-and-add multipliers up to 10 bits, incrementers up to 64 bits, and seeded random MCX circuits of 16, 32 and 64 wires. Hypotheses and analysis were pre-registered; one amendment (cache regimes, timeout censoring, parallel workers) was recorded before any blind measurement was inspected. Workloads: *simulate* (symbolic outputs), *equiv* (against a variant built from sound rewrites) and *nonequiv* (against the circuit with one inserted MCX gate).

Measurement. One fresh worker process per (circuit, workload, mode), pinned to one core of a 4-core Intel Xeon VM (CPython 3.12.3; extension built with GCC 13.3.0 and `-O3`[1]). The two corpora ran concurrently, the QCEC corpus with one worker on one core and the blind corpus with two workers on two other cores, while lighter jobs used the remaining core; shared caches may perturb timings, but every ratio is paired within one run. One warmup run, then the median of 7 runs (3 above 1 s, 1 above 10 s); 120 s per cell; cells of one family run in increasing size and larger ones are censored after a timeout. Ratios are paired per circuit; we report geometric means with 95% percentile-bootstrap intervals over circuits, the range and the number of wins. Explicit monomial extraction is timed once and feeds an output fingerprint compared across all modes. Versions: PolyBoRi through passagemath-brial 10.8.13, CUDD 3.0.0 through `dd` 0.6.0, MQT QCEC 3.11.0 restricted to its complete alternating checker (no simulation or ZX checker, so every conclusive verdict is a proof).

Exactness checks. Beyond the cross-mode fingerprints, the test suite of the measured commit (2 490 tests) compares every backend, the native one included, with concrete execution on all inputs of seeded random circuits of up to 7 wires, checks the native engine against the Python ZDD node for node, and shows that the same differential checks catch deliberately broken Python algebras (missing cross term, no cancellation, OR instead of XOR). Runs of the native tests under AddressSanitizer and UndefinedBehaviorSanitizer were added together with the overflow fix and are not part of the measured commit. These are tests, not proofs: no part of the engine is machine-checked.

[1] The run records list the output of `cc -version` (clang 18), which is not the compiler that built the extension; the measured binary, identified by its SHA-256 in the records, carries GCC 13.3.0 in its `.comment` section.

Table 1: Matched drivers, cold caches, compute phase. Geometric mean of the paired ratio baseline/candidate (> 1: symq faster) with a 95% bootstrap interval; in parentheses, wins/circuits completed by both, and circuits not completed (n/c) by at least one side. Modes: nat-loop: symq native, minimal loop; pb-loop: PolyBoRi, minimal loop; nat-sorted: symq native, minimal loop, sorted; pb-sorted: PolyBoRi, minimal loop, sorted; nat-fused: symq native, fused C loop; pb-best: PolyBoRi, best of its three drivers; bdd-loop: CUDD BDD, minimal loop (reordering); bdd-static: CUDD BDD, minimal loop (static order).

candidate / baseline	simulate	equiv	nonequiv
QCEC (19 circuits)			
nat-loop / pb-loop	4.16 [3.68, 4.67] (19/19)	2.98 [2.59, 3.42] (19/19)	3.28 [2.97, 3.61] (19/19)
nat-sorted / pb-sorted	4.01 [3.43, 4.57] (19/19)	3.00 [2.51, 3.48] (18/19)	3.64 [3.21, 4.10] (19/19)
nat-fused / pb-best	4.01 [3.67, 4.37] (19/19)	2.98 [2.59, 3.36] (19/19)	3.31 [3.12, 3.50] (19/19)
nat-loop / bdd-loop	3.86 [3.08, 5.18] (19/19)	2.62 [2.16, 3.17] (18/19)	3.33 [2.77, 4.00] (19/19)
nat-loop / bdd-static	3.54 [2.99, 4.24] (19/19)	2.58 [2.11, 3.12] (18/19)	3.21 [2.77, 3.73] (19/19)
blind (30 circuits)			
run not available			

Table 2: Ablation of the native engine (cold compute phase). Each row changes one thing: the engine (C instead of Python, same driver), the driver (minimal loop instead of symq's `Algebra` layer), factor sorting, and fusing the loop in C. Ratios below 1 are slowdowns. Modes: nat-alg: symq native, `Algebra` driver; py-alg: symq Python ZDD, `Algebra` driver; nat-loop: symq native, minimal loop; nat-sorted: symq native, minimal loop, sorted; nat-fused: symq native, fused C loop.

candidate / baseline	simulate	equiv	nonequiv
QCEC (19 circuits)			
nat-alg / py-alg	4.92 [4.37, 5.56] (19/19)	4.16 [3.33, 5.40] (19/19)	3.91 [3.12, 4.76] (18/19)
nat-loop / nat-alg	2.18 [1.99, 2.37] (19/19)	2.13 [1.84, 2.49] (19/19)	2.47 [2.12, 3.01] (19/19)
nat-sorted / nat-loop	0.85 [0.81, 0.90] (0/19)	0.89 [0.83, 0.95] (2/19)	0.91 [0.88, 0.95] (3/19)
nat-fused / nat-sorted	1.14 [1.03, 1.27] (13/19)	1.13 [1.05, 1.21] (17/19)	1.10 [1.02, 1.19] (15/19)
nat-fused / py-alg	10.45 [9.39, 11.58] (19/19)	8.92 [7.90, 10.42] (19/19)	9.76 [8.68, 11.15] (19/19)
blind (30 circuits)			
run not available			

6 Results

Exactness (Q4). A group is one (circuit, workload) cell across all symbolic modes. For *simulate* the compared fingerprint is a hash of the explicit output monomials (the polynomials themselves); for *equiv* and *nonequiv* it is only the verdict and the index of the first differing output, and every counterexample is re-checked concretely. On the QCEC corpus, 57 groups were compared with 0 disagreement; QCEC returned 38 correct proofs. [PENDING, not a result: blind-corpus exactness checks.] No counterexample failed its concrete re-check.

Matched comparison (Q1). Table 1 is the main result. With the same minimal loop and cold caches, the native engine is 4.16× faster than PolyBoRi on *simulate* over the QCEC corpus and wins on all 19 circuits; equivalence and non-equivalence checks, which also simulate a second circuit and compare, show 2.98× and 3.28×. With warm caches (Table 4) the ratio drops to 2.11×: part of the cold-cache gap is the cost of filling PolyBoRi's caches. Against CUDD BDDs, which represent functions rather than monomials, the native engine is 3.86× faster (dynamic reordering) on *simulate*. Fixed costs are listed separately (Table 5): a fresh PolyBoRi ring costs 4.108 ms and a CUDD manager 2.659 ms, against 0.010 ms for the native manager. They dominate end-to-end times on these small circuits, which is why we do not headline end-to-end ratios.

Ablation (Q2). Table 2 decomposes the distance between the Python ZDD and the shipped engine. The C core alone, under symq's `Algebra` driver, gives 4.92× cold (but only 1.23× warm, when nearly every operation is a cache hit and the driver dominates). Replacing that driver by the minimal loop gives another 2.18×: symq's per-gate guards and counters are a large share of the cost of small circuits. Sorting factors by monomial count, which our fused loop does, *slows* the engine (0.85×); fusing the loop in C then recovers 1.14×. Measured directly, the shipped fused loop against the unsorted minimal Python loop is 0.97× (95% CI [0.89, 1.06], 9/19 wins): no measurable gain. Our earlier attribution of a 2.65× gain to fusion was therefore the removal of the `Algebra` layer. On these circuits, the per-operation interpreter overhead that fusion removes is about as large as the cost of the sorting that the fused loop adds; we have not tested longer, shallower circuits, where the overhead could matter more.

Table 3: Variable-order sensitivity of the native engine (*simulate*, fused loop, cold): identity, reversed and five seeded random orders per circuit. Node spread: largest over smallest number of ZDD nodes across the orders of one circuit. Fingerprints: hashes of the explicit output monomials, which must not depend on the order.

corpus	complete orders	equal fingerprints	node spread geo. / max	identity/best time (geo.)	identity best
QCEC (19 circuits)	19/19	19/19	1.79 / 146 (`c2_181`)	1.26	6/19
blind (30 circuits)	15/30	15/15	5.36 / 85 (`adder_16`)	3.56	0/15

Table 4: Same comparison with warm caches: one manager reused across runs after a filling run (re-verification of the same circuit). Modes: nat-loop: symq native, minimal loop; pb-loop: PolyBoRi, minimal loop; nat-fused: symq native, fused C loop; pb-best: PolyBoRi, best of its three drivers; bdd-loop: CUDD BDD, minimal loop (reordering).

candidate / baseline	simulate	equiv	nonequiv
QCEC (19 circuits)			
nat-loop / pb-loop	2.11 [2.03, 2.19] (19/19)	2.12 [2.05, 2.18] (19/19)	2.15 [2.07, 2.24] (19/19)
nat-fused / pb-best	2.14 [1.83, 2.52] (19/19)	2.23 [1.95, 2.54] (19/19)	2.22 [1.89, 2.61] (19/19)
nat-loop / bdd-loop	2.46 [2.27, 2.72] (19/19)	2.58 [2.37, 2.85] (19/19)	2.63 [2.35, 3.03] (19/19)
blind (30 circuits)			
run not available			

Variable order (robustness). Table 3 varies the variable order of the native engine (runs `e71c7e0fc8a2`; identity, reversed and five seeded random orders; 25 s and $2 \cdot 10^7$ nodes per cell; family-wise censoring on the blind corpus). The output monomials are identical under every order that completed, as they must be (0 and 0 disagreements). On the QCEC corpus the default (identity) order is within 1.26× of the best of the seven in time (geometric mean) and the best on 6/19 circuits, although node counts vary by up to 146× (`c2_181`). The blind corpus is different: the identity order is never the best, is 3.56× slower than the best order on the 15 circuits where all orders complete, up to 164.4× on `adder_16`, and exhausts the budget on `adder_24`, `adder_31`, `cadder_24`, `cadder_30` where some random order completes. The adders that fail under the default order are thus an *order* effect, not a limit of the representation; the multipliers beyond 5 bits and the dense random circuits fail under all seven orders or are censored after a smaller circuit of their family failed (11 circuits). In the matched comparison every monomial engine uses the index order (PolyBoRi's ring too) and CUDD's dynamic reordering is the only adaptive ordering measured.

Blind corpus (Q3). Table 6 gives the completion of every mode, Table 7 the paired ratios by family and Table 1 the aggregates. Ratios are computed only on circuits both tools complete, a selection that can favour either side; coverage is reported next to them. [PENDING, not a result: the blind-corpus run (frozen corpus, pre-registered hypotheses H1–H4) had not finished when this draft was compiled.] [PENDING, not a result: blind-corpus coverage.] [PENDING, not a result: interpretation of the sealed blind numbers, to be written by hand after sealing.]

Deviations from the pre-registration. The corpus, the hypotheses and the amendment were pushed to the public repository at 14:57:38 and 15:06:11 UTC (GitHub activity log), before the blind run started at 15:06:21 UTC. The following departures happened afterwards. (1) The progress log of the run printed the status of every cell (never its times), and we read it while the run was going; the variable-order experiment on the blind corpus, not pre-registered and therefore exploratory, was run and inspected at the same time. (2) H4 names no test; we report the counts above and draw no verdict. (3) Censoring grouped the random circuits by name prefix, so one timeout on 16 wires censored the 32- and 64-wire random circuits with other gate counts; censored cells are not failures. (4) A worker killed by the kernel for lack of memory was recorded as *error* without its signal and did not censor larger circuits; the kernel log records out-of-memory kills of 7–13 GiB worker processes during the run and no crash (`paper/evidence/blind_oom.txt`). (5) The fused mode of this run built its index arrays and gathered statistics inside the timed phase (Section 5 describes the corrected harness); H2 is therefore also evaluated on a supplementary *simulate* run with the corrected harness. (6) The run shared the machine with other measurements. Items (3)–(5) are fixed in the harness for future runs.

Table 5: Fixed and conversion costs, QCEC corpus, *simulate* (median over circuits of the per-circuit medians, milliseconds). Setup: manager and variables (PolyBoRi: a fresh ring). Extraction: explicit monomials of all outputs.

mode	setup	compute	extraction
symq native, fused C loop	0.010	0.037	0.105
symq native, minimal loop	0.010	0.037	0.111
PolyBoRi, minimal loop	4.108	0.191	0.593
CUDD BDD, minimal loop (reordering)	2.659	0.133	2.680
symq Python ZDD, `Algebra` driver	0.019	0.509	0.198
symq sparse ANF (Python)	0.007	0.186	0.108

Table 6: Completion on the blind corpus (90 cells per mode: 30 circuits × 3 workloads; QCEC has no *simulate*). *censored*: not run because a smaller circuit of the same family timed out (120 s).

mode	ok	timeout	censored	error
run not available				

MQT QCEC. Table 8 reports QCEC for context, end to end on both sides (conversion and check for QCEC; manager setup, both simulations, comparison and witness for symq). On the QCEC corpus the ratio is 32.50× on *equiv* (95% CI [21.34, 50.63]). No QCEC verification takes less than 1.7 ms, while the median native one takes 0.10 ms: on these small circuits the ratio mostly measures QCEC's fixed cost and says nothing about larger circuits. The comparison is not matched: QCEC decides unitary equality for general quantum circuits and builds no polynomial, and its default portfolio, which we disabled to keep only complete proofs, may be faster.

7 Threats to validity and limitations

- **Single machine, small circuits.** Medians are often below one millisecond; one VM; no CPU frequency control. Paired ratios and bootstrap intervals over circuits do not remove machine effects.
- **Drivers and bindings.** The minimal loop is Python for every engine, and the bindings differ in per-call cost. We calibrate that cost and bound its effect (Section 6), but a native driver for PolyBoRi (its C++ API) or CUDD (its C API) on the same gate arrays is the only clean answer and is future work.
- **Memory and caches.** The native engine never evicts its operation cache and never collects garbage; CUDD runs with garbage collection and `dd`'s default cache limits; PolyBoRi with its defaults. We report no memory comparison (the recorded peak-RSS growth is not comparable across tools), and in the blind corpus a native cell can die from memory exhaustion rather than time out. Part of the cold/warm difference may be page faults and lazy initialisation inside the timed phase rather than cache contents.
- **Missing baselines.** We compare with no SAT-based combinational equivalence checker (e.g. ABC's `cec` [**?**] on a miter) and no path-sum verifier (Feynman [1]); both are natural answers to the cases where ANF explodes.
- **Configuration.** PolyBoRi uses the `lp` order; CUDD its defaults, with and without dynamic reordering; no tuned variable order for anyone.
- **Representation and order.** Width at most 64; monomial representations explode on multipliers (Tables 6 and 3); the default variable order fails on adders that other orders handle, and we have no ordering heuristic; no garbage collection, so memory grows with the number of operations of a verification.
- **Corpus.** The QCEC circuits influenced the design; the blind corpus is generated, not gathered from independent sources. Many QCEC test circuits come from RevLib; we rely on the licence of the QCEC repository and have not traced each file's provenance.
- **Scope.** Classical reversible gates only. symq also contains a specification and reference implementation of a restricted phase fragment (±1 phases, X-basis measure-and-reset, single-outcome Z feed-forward)

Table 7: Blind corpus by family, *simulate*, cold compute phase: geometric mean of the paired ratio (completed by both / circuits in the family). Last column: circuits completed by the native minimal loop but not PolyBoRi, by PolyBoRi but not the native loop, and by CUDD but not the native loop.

family	vs. pb-loop	vs. bdd-loop	nat\pb / pb\nat / bdd\nat
run not available			

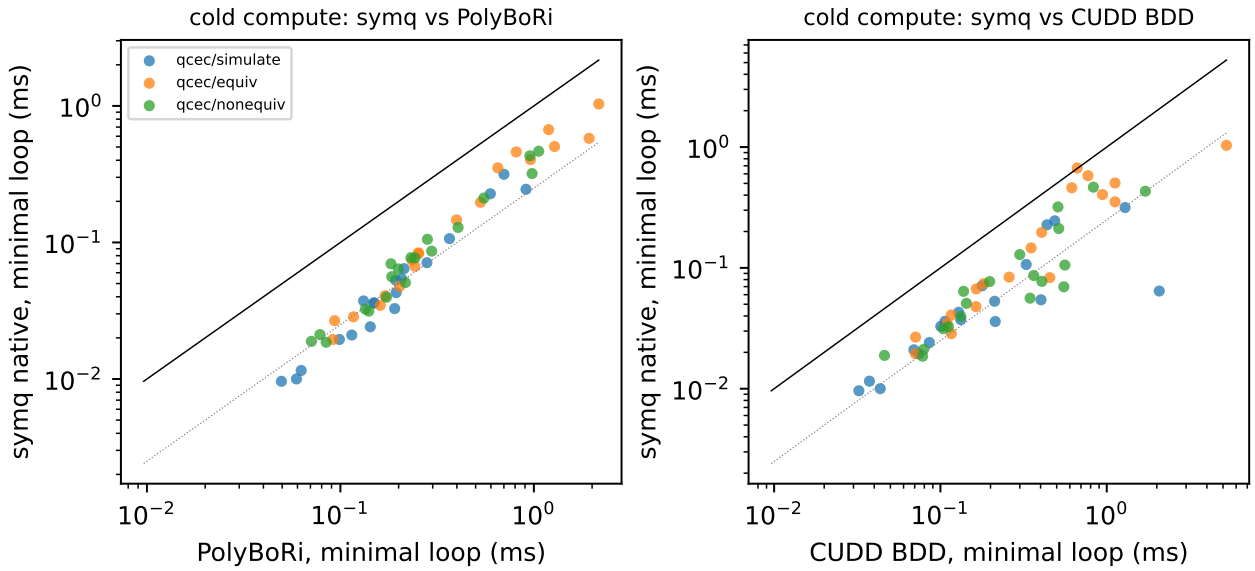


Figure 1: Per-circuit cold compute time of the native engine against PolyBoRi (left) and CUDD BDD (right), same minimal loop, all workloads, circuits completed by both. A constant factor over two orders of magnitude of running time is what a per-call cost would produce, which is why we calibrate the bindings separately. Solid: equal time; dotted: 4× faster.

aimed at measurement-based uncomputation [11]; it has proof sketches and exhaustive small-case tests against an independent state-vector oracle, but no external corpus and no budgeted backend, and is not a contribution of this paper. Outcome-complete phase simulation and hybrid-circuit checkers [18, 24, 16] address related, broader questions.

8 Conclusion

A specialised native monomial-ZDD engine checks the equivalence of small reversible circuits several times faster than PolyBoRi and CUDD when every tool is driven identically and caches are controlled; most of what looked like the benefit of a fused C loop was the removal of our own driver overhead, and sorting factors by size hurts. These are engineering results on one machine and two small corpora, not a claim of algorithmic novelty or general superiority.

Artifact and reproducibility

Code, corpora, sealed results (with SHA-256 manifests) and the scripts that generate every table and figure of this paper are in the `symq` repository; `paper/REPRODUCIBILITY.md` gives the commands. The QCEC corpus run is `23469829a0dc`, the blind corpus run [PENDING, not a result: not sealed yet], the variable-order run `e71c7e0fc8a2`; all use the same native binary (SHA-256 prefix `2e3f6791`). [TODO: archive the exact commit (tag and DOI) after author approval; no release has been made.]

Acknowledgements and disclosure

[TODO: authors to write; must disclose the use of AI assistance in code, experiments and drafting in accordance with arXiv policy, and any competing interests.]

Table 8: End-to-end time of one verification: symq native (manager setup, both simulations, comparison, witness) against MQT QCEC's alternating decision-diagram checker (circuit conversion and check, default construction of its own managers). QCEC answers unitary equality and builds no polynomial; *simulate* does not apply. Modes: nat-fused: symq native, fused C loop; qcec: MQT QCEC (alternating checker).

candidate / baseline	simulate	equiv	nonequiv
QCEC (19 circuits)			
nat-fused / qcec	n/a	32.50 [21.34, 50.63] (19/19)	38.35 [27.49, 51.06] (19/19)
blind (30 circuits)			
run not available			

References

[1] Matthew Amy. Towards large-scale functional verification of universal quantum circuits. In *Proceedings 15th International Conference on Quantum Physics and Logic (QPL 2018)*, volume 287 of *Electronic Proceedings in Theoretical Computer Science*, pages 1–21, 2019.

[2] Matthew Amy, Dmitri Maslov, and Michele Mosca. Polynomial-time T-depth optimization of Clifford+T circuits via matroid partitioning. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 33(10):1476–1489, 2014.

[3] Michael Brickenstein and Alexander Dreyer. PolyBoRi: A framework for Gröbner-basis computations with Boolean polynomials. *Journal of Symbolic Computation*, 44(9):1326–1345, 2009.

[4] Michael Brickenstein, Alexander Dreyer, Gert-Martin Greuel, Markus Wedler, and Oliver Wienand. New developments in the theory of Groebner bases and applications to formal verification. arXiv:0801.1177 [math.AC], 2008.

[5] Randal E. Bryant. Graph-based algorithms for Boolean function manipulation. *IEEE Transactions on Computers*, C-35(8):677–691, 1986.

[6] Lukas Burgholzer and Robert Wille. Advanced equivalence checking for quantum circuits. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 40(9):1810–1824, 2021.

[7] Lukas Burgholzer and Robert Wille. QCEC: A JKQ tool for quantum circuit equivalence checking. *Software Impacts*, 7:100051, 2021.

[8] Lukas Burgholzer and Robert Wille. Handling non-unitaries in quantum circuit equivalence checking. In *Proceedings of the 59th ACM/IEEE Design Automation Conference (DAC)*, pages 529–534, 2022.

[9] California Institute of Technology. dd: Binary decision diagrams in pure Python and Cython wrappers of CUDD, Sylvan, and BuDDy. `https://github.com/tulip-control/dd`, 2023. Version 0.6.0, BSD-3-Clause.

[10] Steven A. Cuccaro, Thomas G. Draper, Samuel A. Kutin, and David Petrie Moulton. A new quantum ripple-carry addition circuit. arXiv:quant-ph/0410184, 2004.

[11] Craig Gidney. Halving the cost of quantum addition. *Quantum*, 2:74, 2018.

[12] Xin Hong, Yuan Feng, Sanjiang Li, and Mingsheng Ying. Equivalence checking of dynamic quantum circuits. In *Proceedings of the 41st IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, pages 1–8, 2022.

[13] Wei-Jia Huang, Christophe Chareton, Yu-Fang Chen, Kai-Min Chung, Min-Hsiu Hsieh, Alfons Laarman, and Jingyi Mei. Equivalence checking of quantum circuits via path-sum and weighted model counting. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, Lecture Notes in Computer Science, pages 419–439. Springer, 2026.

[14] Daniela Kaufmann and Jérémy Berthomieu. Extracting linear relations from Gröbner bases for formal verification of and-inverter graphs. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, LNCS, pages 355–374, 2025.

[15] Daniela Kaufmann and Armin Biere. Improving AMulet2 for verifying multiplier circuits using SAT solving and computer algebra. *International Journal on Software Tools for Technology Transfer*, 25(2):133–144, 2023.

[16] Jingyu Ke, Jingyang Li, and Guoqiang Li. Irene: Equivalence checking of hybrid quantum programs via structure-preserving symbolic reduction. arXiv:2609.36065 [cs.PL], 2026.

[17] Vadym Kliuchnikov, Michael Beverland, and Adam Paetznick. Stabilizer circuit verification. arXiv:2309.08676 [quant-ph], 2023.

[18] Vadym Kliuchnikov, Adam Paetznick, and Marcus P. da Silva. Phased outcome-complete simulation. arXiv:2603.24717 [quant-ph], 2026.

[19] Alireza Mahzoon, Daniel Große, and Rolf Drechsler. RevSCA-2.0: SCA-based formal verification of nontrivial multipliers using reverse engineering and local vanishing removal. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 41(5):1573–1586, 2022.

[20] Shin-ichi Minato. Zero-suppressed BDDs for set manipulation in combinatorial problems. In *Proceedings of the 30th ACM/IEEE Design Automation Conference (DAC)*, pages 272–277. ACM, 1993.

[21] Shin-ichi Minato. Implicit manipulation of polynomials using zero-suppressed BDDs. In *Proceedings of the European Design and Test Conference (ED&TC)*, pages 449–454, 1995.

[22] Shin-ichi Minato. Zero-suppressed BDDs and their applications. *International Journal on Software Tools for Technology Transfer*, 3(2):156–170, 2001.

[23] Munich Quantum Toolkit. MQT QCEC: A tool for quantum circuit equivalence checking. `https://github.com/munich-quantum-toolkit/qcec`, 2026. Version 3.11.0, MIT license; test circuits taken from commit 98d6784.

[24] Jérome Ricciardi, Sébastien Bardin, Christophe Chareton, and Benoît Valiron. Quantum circuit equivalence checking: A tractable bridge from unitary to hybrid circuits. arXiv:2511.22523 [quant-ph], 2025.

[25] Fabio Somenzi. *CUDD: CU Decision Diagram Package, Release 3.0.0*. University of Colorado at Boulder, 2015. Mirror: `https://github.com/cuddorg/cudd`, tag cudd-3.0.0.

[26] The BRiAl developers. BRiAl: successor to PolyBoRi. `https://github.com/BRiAl/BRiAl`, 2025. Version 1.2.15, GPL-2.0-or-later.

[27] The passagemath developers. passagemath-brial: Boolean polynomials via BRiAl, modularized SageMath distribution. `https://pypi.org/project/passagemath-brial/`, 2026. Version 10.8.13.

[28] George F. Viamontes, Igor L. Markov, and John P. Hayes. Checking equivalence of quantum circuits and states. In *IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, pages 69–74, 2007.

[29] Ziyuan Wang, Bin Cheng, Longxiang Yuan, and Zhengfeng Ji. FeynmanDD: Quantum circuit analysis with classical decision diagrams. In *Computer Aided Verification (CAV)*, volume 15934 of *LNCS*, pages 28–52, 2025.

[30] Robert Wille, Daniel Große, D. Michael Miller, and Rolf Drechsler. Equivalence checking of reversible circuits. In *39th International Symposium on Multiple-Valued Logic (ISMVL)*, pages 324–330, 2009.

[31] Robert Wille, Daniel Große, Lisa Teuber, Gerhard W. Dueck, and Rolf Drechsler. RevLib: An online resource for reversible functions and reversible circuits. In *38th International Symposium on Multiple Valued Logic (ISMVL)*, pages 220–225, 2008.